

CS102L Data Structures & Algorithms

Lab 6



DEPARTMENT OF COMPUTER SCIENCE
DHANANI SCHOOL OF SCIENCE AND ENGINEERING
HABIB UNIVERSITY
SPRING 2026

Contents

1	Introduction	3
1.1	Restriction	3
1.2	Instructions	3
1.3	Marking scheme	3
1.4	Lab Objectives	3
1.5	Late submission policy	4
1.6	Use of AI	4
1.7	Viva	4
2	Prerequisites	5
2.1	Assert Statements	5
2.2	Pytest	5
3	Exercises	7
3.1	Binary Search Iterative	7
3.1.1	Function Description	7
3.1.2	Examples	7
3.1.3	Testing	7
3.2	Binary Search Iterative Modified	7
3.2.1	Function Description	8
3.2.2	Examples	8
3.2.3	Testing	8
3.3	Binary Search Recursive	8
3.3.1	Examples	9
3.3.2	Testing	9
3.4	Binary Search Recursive Modified	9
3.4.1	Function Description	9
3.4.2	Examples	9
3.4.3	Testing	10
3.5	Finding Multiple	10
3.5.1	Function Description	10
3.5.2	Examples	10
3.5.3	Testing	10
3.6	Updating Student Record	10
3.6.1	Function Description	11
3.6.2	Examples	11

3.6.3	Testing	11
-------	-------------------	----

Introduction

1.1 Restriction

You are not allowed to use any built in functions other than the append and insert function

1.2 Instructions

- This lab will contribute 1% towards your final grade.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *lab6-aa12345.zip* where *aa12345* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.3 Marking scheme

This lab will be marked out of 100.

- 50 Marks are for the completion of the lab.
- 50 Marks are for progress and attendance during the lab.

1.4 Lab Objectives

- To understand and implement binary search in Python
- To use binary search in different exercises

1.5 Late submission policy

There is no late submission policy for the lab.

1.6 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.7 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an `AssertionError` is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with `test_` in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py .....
===== 10 passed in 0.06s =====
```

However, if all the tests fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFFFF

===== FAILURES =====
----- test_question1[1st10-1st20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] -----
```

Exercises

3.1 Binary Search Iterative

This function should be implemented in the file `q1.py`.

3.1.1 Function Description

Write a function named `binary_search_iterative`, which takes the following two parameters:

- `lst` – A sorted list of elements, sorted in ascending order.
- `item` – An item to search for in the list.

The function should return the index of `item` in `lst` using an iterative binary search approach. If the item is not found, the function should return `-1`.

3.1.2 Examples

```
>>> binary_search_iterative([0, 1, 2, 8, 13, 17, 19, 32, 42], 3)
-1
>>> binary_search_iterative([0, 1, 2, 8, 13, 17, 19, 32, 42], 17)
5
```

3.1.3 Testing

To test your function, type the following command on the terminal:

```
pytest tests/test_q1.py
```

3.2 Binary Search Iterative Modified

This function should be implemented in the file `q2.py`.

3.2.1 Function Description

Write a function named `binary_search_iterative_modified`, which takes the following two parameters:

- `lst` – A sorted list of elements, sorted in ascending order.
- `item` – An item to search for in the list.

The function should return the index of `item` in `lst` using an iterative binary search approach. If the item is not found, it should be inserted into `lst` in the correct position to maintain the sorted order, and its index should be returned.

3.2.2 Examples

```
>>> binary_search_iterative_modified([0, 1, 2, 8, 13, 17,
    19, 32, 42], 8)
3
>>> binary_search_iterative_modified([0, 1, 2, 3, 8, 13, 17,
    19, 32, 42], -1)
0
```

3.2.3 Testing

To test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 Binary Search Recursive

This function should be implemented in the file `q3.py`.

Write a function named `binary_search_recursive`, which takes the following parameters:

- `lst` – A sorted list of elements, sorted in ascending order.
- `item` – An item to search for in the list.
- `low` – The index of the first element of the current search range.
- `high` – The index of the last element of the current search range.

The function should return the index of `item` in `lst` using a recursive binary search approach. If the item is not found, the function should return `-1`.

3.3.1 Examples

```
>>> binary_search_recursive([0, 1, 2, 8, 13, 17, 19, 32, 42], 19, 0, 8)
6
>>> binary_search_recursive([0, 1, 2, 8, 13, 17, 19, 32, 42], 3, 0, 8)
-1
```

3.3.2 Testing

To test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```

3.4 Binary Search Recursive Modified

This function should be implemented in the file `q4.py`.

3.4.1 Function Description

Write a function named `binary_search_recursive_modified`, which takes the following parameters:

- `lst` – A sorted list of elements, sorted in ascending order.
- `item` – An item to search for in the list.
- `low` – The index of the first element of the current search range.
- `high` – The index of the last element of the current search range.

The function should return the index of `item` in `lst` using a recursive binary search approach. If the item is not found, it should be inserted into `lst` in the correct position to maintain the sorted order, and its index should be returned.

3.4.2 Examples

```
>>> binary_search_recursive_modified([0, 1, 2, 8, 13, 17, 19, 32, 42], 3, 0, 8)
3
>>> binary_search_recursive_modified([0, 1, 2, 8, 13, 17, 19, 32, 42], 17, 0, 8)
5
```

3.4.3 Testing

To test your function, type the following command on the terminal:

```
pytest tests/test_q4.py
```

3.5 Finding Multiple

This function should be implemented in the file **q5.py**.

3.5.1 Function Description

Using binary and linear search approaches, write a function named **finding_multiple**, which takes the following parameters:

- **lst** – A sorted list of elements, sorted in ascending order.
- **item** – An item to search for in **lst**.

The function should return a list of all the indices where **item** is found in **lst**. If the item is not found, the function should return an empty list. You must use binary search to locate the item first, and then linear search to find all occurrences of the item in both directions from that position.

3.5.2 Examples

```
>>> finding_multiple([0, 1, 2, 8, 13, 17, 17, 17, 17, 19, 32, 42], 17)
[5, 6, 7, 8]
>>> finding_multiple([0, 1, 2, 8, 13, 17, 17, 17, 17, 19, 32, 42], 34)
[]
```

3.5.3 Testing

To test your function, type the following command on the terminal:

```
pytest tests/test_q5.py
```

3.6 Updating Student Record

This function should be implemented in the file **q6.py**.

3.6.1 Function Description

Using the binary search approach, write a function named `update_record`, which takes the following parameters:

- `student_records` – A list of tuples, sorted in ascending order by student ID. Each tuple of the list represents the student's data in the format *(ID, Email, Mid1, Mid2)*.
- `ID` – The ID of the student whose record needs to be updated.
- `record_title` – The field that must be updated. It will be one of "ID", "Email", "Mid1" or "Mid2".
- `data` – The new value that should replace the existing value in the specified field.

The function should locate the student record associated with the given ID using binary search, update the specified field with the new data, and return "Record updated".

If 'ID' is given as the record_title, return "ID cannot be updated" without making any changes.

If the given ID is not found in the students_records, return "Record not found".

3.6.2 Examples

```
student_records = [("aa02822", "ea02822", 80, 65),
                   ("ea02822", "ea02822@st.habib.edu.pk", 80, 65),
                   ("fa08877", "fa08877@st.habib.edu.pk", 66, 67),
                   ("gh04588", "gh04588@st.habib.edu.pk", 33, 50)]

>>> update_record(student_records, "aa02822", "ID", "
aa02456")
ID cannot be updated

>>> update_record(student_records, "aa02822", "Email", "
aa02822@st.habib.edu.pk")
Record updated

>>> update_record(student_records, 'f08877', "Mid2", 50)
Record not found
```

3.6.3 Testing

To test your function, type the following command on the terminal:

```
pytest tests/test_q6.py
```